一、思考题

1. Thinking 1.1 请阅读 附录中的编译链接详解，尝试分别使用实验环境中的原生 x86 工具 链（gcc、ld、readelf、objdump 等）和 MIPS 交叉编译工具链（带有 mips-linux-gnu 前缀），重复其中的编译和解析过程，观察相应的结果，并解释其中向 objdump 传入的参数 的含义。

objdump中参数

-D：从objfile中反汇编所有的section；

-S：尽可能反汇编出源代码，尤其当编译的时候指定了-g这种调试参数时，效果比较明显。

> **Thinking 1.2** 思考下述问题：
>
> - 尝试使用我们编写的 `readelf` 程序，解析之前在 `target` 目录下生成的内核 ELF 文件。
> - 也许你会发现我们编写的 `readelf` 程序是不能解析 `readelf` 文件本身的，而我们刚才介绍的系统工具 `readelf` 则可以解析，这是为什么呢？(提示：尝试使用 `readelf -h`，并阅读 `tools/readelf` 目录下的 `Makefile`，观察 `readelf` 与 `hello` 的不同)

原因：内核文件是大端存储，但readelf程序文件解析的是小端存储的。

> **Thinking 1.3** 在理论课上我们了解到，MIPS 体系结构上电时，启动入口地址为 `0xBFC00000`（其实启动入口地址是根据具体型号而定的，由硬件逻辑确定，也有可能不是这个地址，但一定是一个确定的地址），但实验操作系统的内核入口并没有放在上电启动地址，而是按照内存布局图放置。思考为什么这样放置内核还能保证内核入口被正确跳转到？
>
> （提示：思考实验中启动过程的两阶段分别由谁执行。）

当操作系统启动时，首先会启动bootloader程序。在MIPS处理器上，当MIPS架构上电时，启动的入口地址是0xBFC00000，这是一个虚拟地址，位于kseg1中。在stage1阶段，虚拟地址的高三位被清零，对应的物理地址是0x1FC00000。从这个物理地址开始执行MIPS的第一条指令，完成基本硬件设备的初始化，并为stage2做准备。

接下来进入stage2阶段的入口函数。当BIOS从MBR中读取启动信息时，以Linux中的GRUB引导管理程序为例，BIOS会加载MBR中的GRUB代码，然后将控制权交给GRUB。GRUB会逐步加载自身代码，识别文件系统，并将文件系统中的内核镜像文件加载到内存中。在这个过程中可能会涉及到内存的移动操作。最终，BIOS会跳转到指定的地址开始执行，确保内核入口得到正确的跳转。

二、难点分析

在本次实验中，主要目标之一是了解ELF文件的结构和功能。然而，在1.3题中遇到了问题，尤其是在start.S文件中设置堆栈点的指令方面。此外，对于gxemul仿真器和Makefile的理解也有待加强，这使得实验进行起来有些困难。

通过加强对ELF文件结构、gxemul和Makefile的理解，相信能够更好地完成实验任务并解决遇到的问题。

三、实验体会

深刻认识到Makefile的使用和make命令的重要性，还掌握了内核地址和MIPS组合的新知识。 虽然已经了解了相对较长时间的lab1，但与花费的时间和努力相比，学习学习量仍然不多。 理解代码整体流动的能力相对增强，对Head文件之间的相互调用有了新的了解。 虽然对专业知识还很不足的我来说感觉很难，但是能学习这样的东西，非常感谢。